

PRODUCT REQUIREMENTS DOCUMENT

AI Startup Launch Team

An orchestrated multi-agent platform that confronts founders with brutally honest, data-anchored validation — replacing blind AI confirmation bias with structural realism.

DOCUMENT VERSION	1.0 — Draft for Engineering Review
DATE	September 13, 2026
PREPARED BY	Principal Technical Product Manager, AI Systems Architecture
CLASSIFICATION	Internal — Engineering & Product
STACK	FastAPI · LangChain · Supabase (PostgreSQL) · Next.js 14 · Vercel · Render



Table of Contents

- 1. Executive Summary & Product Vision
- 2. Deep Inter-Agent Architecture & Flow Description
- 3. Detailed Sub-Agent Specifications
 - 3.1 Chief AI Orchestrator Agent
 - 3.2 Market Research Agent
 - 3.3 Product Strategist Agent
 - 3.4 Pricing Agent
 - 3.5 Marketing Agent
- 4. Technical Architecture, Schema & Folder Tree
- 5. Non-Functional Requirements & Security
- 6. Appendix — Glossary & Assumptions

1. Executive Summary & Product Vision

1.1 Core Product Thesis

Most AI-powered "startup idea validator" tools are built to flatter. They take a vague concept and return an optimistic, generic business plan that confirms whatever the founder already believed. **AI Startup Launch Team** is built on the opposite premise: an orchestrated, multi-agent AI consultancy that treats validation as an adversarial, evidence-driven process. The system's job is not to make founders feel good — it is to stop them from building the wrong thing, by forcing every claim through live market data, historical failure precedent, and hard unit-economics math before a single feature is scoped.

The platform packages four specialist sub-agents (Market Research, Product Strategy, Pricing, and Marketing) behind a single Chief Orchestrator Agent that acts as a skeptical editor-in-chief: cross-questioning weak inputs, flagging contradictions between sub-agent outputs, and — where the data supports it — citing the exact historical startup failures (Kozmo, WeWork, Fast, Juicero) that match the founder's specific structural risk.

1.2 Vision Statement

To become the default "reality-check layer" every early-stage founder runs their idea through before writing a line of code or a single ad — an AI team that behaves like a brutally honest co-founder with access to real-time market intelligence, not a hype machine.

1.3 Target Audience Persona

Attribute	Description
Primary Persona	"The Excited First-Timer" — a solo or small-team founder (student, indie hacker, career-switcher) with a strong conviction about an idea, limited go-to-market experience, and a tendency toward confirmation bias when using generic AI tools.
Secondary Persona	"The Serial Validator" — a repeat builder who runs multiple ideas per quarter and wants a fast, structured kill/pursue signal rather than a slow manual research cycle.
Core Pain Point	Existing AI chat tools over-validate; founders waste months and capital building products nobody asked for, priced with no unit-economics grounding, and marketed with no channel strategy.
Jobs-to-be-Done	"Tell me, with evidence, whether this idea is structurally sound — and if it isn't, tell me exactly why and who else failed this way."

1.4 Monetization Strategy & Access Controls

Model: One-time premium transaction engine at **\$20 per full validation run** (not a recurring subscription at MVP). Each paid run unlocks one complete Orchestrator-to-Dashboard pipeline execution (all 4 sub-agents + compiled report). This keeps the funnel low-friction for entrepreneurs testing a single idea while establishing a hard cost gate that filters low-intent traffic.

- Free Tier:** Access to the onboarding wizard and a single locked preview of the Orchestrator's "Low-Information Warning" and a redacted one-paragraph executive summary. Full dashboard, resources, and sub-agent JSON payloads remain gated.
- Paid Tier (\$20 / run):** Full pipeline execution, complete dashboard (Market Research, Product Strategy, Pricing, Marketing), persisted workspace history, downloadable PDF report, and re-run capability with modified inputs at a discounted re-run rate (configurable, default 50% off within 14 days).
- Access Control:** Enforced via Supabase Row-Level Security (RLS) policies keyed to ``auth.uid()``; payment confirmation (Stripe webhook) flips a ``workspaces.is_unlocked`` boolean before the orchestrator pipeline is allowed to execute past the free preview stage.
- Abuse Prevention:** Rate-limited free-preview generation per account/IP (max 3/day) to prevent scraping the Orchestrator's warning logic without payment.

2. Deep Inter-Agent Architecture & Flow Description

2.1 End-to-End User Traversal Path

Step 1 — Multi-Wizard Onboarding (Next.js UI): The user progresses through a stepped wizard collecting all fields required by the five downstream consumers (Core Idea, Target Industry, Target Location, Audience Demographics, Known Competitors, Must-Have Feature, Delivery Mechanism, Visual Vibe, Tech Stack, Customer Segment, CAC Plan, Profit Expectation, Tone of Voice, Budget, Timeline). Client-side Zod validation blocks empty-state submission.

Step 2 — Persistence in Supabase: On each wizard step transition, a debounced PATCH writes partial state into `onboarding_inputs` keyed by `workspace_id`, so no progress is lost on refresh or tab close. Final submission flips `workspaces.status` to `pending_validation`.

Step 3 — Orchestrator Pre-Flight Validation: The FastAPI backend loads the full onboarding record, and the Chief Orchestrator Agent performs an **Input Validation & Guard** pass before any sub-agent is invoked. If inputs are vague or low-effort, it emits a `low_information_warning` flag and (in the paid flow) still proceeds but visibly annotates downstream confidence scores as reduced.

Step 4 — Sub-Agent Pipeline Execution: Market Research runs first and is **blocking** (its output feeds every other agent). Product Strategist and Pricing run **in parallel** once Market Research resolves (Pricing needs MVP feature cost shape from the Strategist for COGS estimation, so Pricing is scheduled slightly after Strategist begins streaming its Must-Have feature list — implemented as a dependent async task, not a hard sequential block). Marketing runs last, consuming Strategist + Pricing outputs.

Step 5 — Orchestrator Synthesis: All four structured JSON payloads are compiled by the Orchestrator into a unified `compiled_dashboard` object, cross-referenced against the historical failure-case vector index, and a final `validation_status` (Green / Amber / Red) is computed.

Step 6 — Dynamic UI Dashboard Visualization: The Next.js dashboard renders the compiled payload as distinct panels (Reality Check banner, Market Intelligence, Product Roadmap, Pricing Model, GTM Plan) with the verified resource links surfaced as clickable citations.

2.2 Execution Topology (Sequential vs Parallel)

Stage	Agent(s)	Mode	Depends On
1	Chief Orchestrator (pre-flight)	Sequential	Onboarding record
2	Market Research Agent	Sequential (blocking)	Orchestrator pre-flight pass
3a	Product Strategist Agent	Parallel branch A	Market Research output
3b	Pricing Agent	Parallel branch B (delayed start)	Market Research output + partial Strategist MVP shape
4	Marketing Agent	Sequential	Strategist output + Pricing output
5	Chief Orchestrator (synthesis)	Sequential	All 4 sub-agent JSON payloads

2.3 Error Handling Boundaries

Every sub-agent call is wrapped in a strict JSON-schema validation boundary before its output is trusted by the Orchestrator or persisted to `agent_runs`.

Failure Mode	System Response
Sub-agent returns malformed / non-JSON output	Single automatic retry with a corrective re-prompt appending the exact Pydantic validation error. On second failure, the agent run is marked <code>failed</code> , and the Orchestrator substitutes a degraded-mode placeholder object with <code>"status": "unavailable"</code> so the pipeline does not fully collapse.
Sub-agent JSON is valid but violates business rules (e.g., Pricing Agent returns <code>LTV/CAC < 3x</code>)	Not treated as a technical error — this is exactly the kind of red flag the product exists to surface. The Orchestrator ingests it as-is and elevates <code>validation_status</code> toward Amber/Red with the specific rule violation cited.
SerpApi timeout or rate-limit (Market Research)	Exponential backoff (3 attempts, 2s/4s/8s). On exhaustion, Market Research returns a partial payload flagged <code>data_confidence: "low"</code> ; downstream agents are informed via a system-level annotation so they do not silently treat sparse data as authoritative.
Full pipeline timeout (> configured SLA, default 120s)	Backend returns a <code>202 Accepted</code> immediately at submission; the actual pipeline runs as a background worker (Render worker dyno) and the frontend polls / subscribes via Supabase Realtime on <code>agent_runs</code> row updates, so long-running LLM calls never hit an HTTP timeout wall.
Payment confirmed but pipeline	

crashes post-charge	Idempotent retry endpoint keyed by <code>workspace_id</code> + Stripe <code>payment_intent_id</code> ; the run is automatically re-queued once at no additional charge before support escalation.
---------------------	---

3. Detailed Sub-Agent Specifications

Every agent below is documented against five mandatory dimensions: functional scope, complete input matrix, exact production system prompt, internal tooling/execution engine, and the strict output JSON schema returned to the Orchestrator.

3.1 Chief AI Orchestrator Agent

CENTRAL BRAIN GUARDRAIL ENGINE HISTORICAL DATA LINKER

Functional Scope

The Orchestrator is the only agent with visibility into the full pipeline state. It solves two problems: (1) preventing garbage-in-garbage-out by gating vague inputs before expensive sub-agent calls are made, and (2) synthesizing four independent structured outputs into one coherent, evidence-anchored verdict — including matching the founder's specific risk pattern against known historical startup collapses.

Complete Input Matrix

Field	Source	Type
Full onboarding record (all wizard fields)	onboarding_inputs table	JSON object
Market Research Agent output	Agent pipeline	JSON payload
Product Strategist Agent output	Agent pipeline	JSON payload
Pricing Agent output	Agent pipeline	JSON payload
Marketing Agent output	Agent pipeline	JSON payload
Historical failure-case vector index	Internal vector store (curated dataset)	Embedding matches

System Prompt (Production)

You are the Chief AI Orchestrator for an elite Startup Launch Consultancy Team. Your job is to act as the central brain, input validator, and final synthesizer for the user's startup idea. You operate with brutal honesty and structural realism. You never blindly validate a weak or vague concept.

CRITICAL OPERATIONAL RULES:

- INPUT VALIDATION & GUARD: If the user provided vague, low-effort inputs (e.g., "I want to build a new Instagram"), you must explicitly trigger a "Low-Information Warning" flag. Warn the user that insufficient data degrades the validation quality.
- HISTORICAL REALITY CHECK: If the sub-agent data (Market Research or Pricing) reveals high risk, you must cross-reference historical tech startup failures (e.g., Kozmo, WeWork, Fast, Juicero) matching the operational bottleneck and present the exact reasons for their collapse.
- AUTHENTICITY & TRUST: Every insight must be anchored to verified facts. You must output a "Resources" section containing the raw URLs and reference links provided by the Market Research sub-agent.
- AGENT ROUTING SYNTHESIS: Compile the JSON payloads from the Market Research, Product Strategist, Pricing, and Marketing sub-agents into a unified, high-integrity executive dashboard response.

EXPECTED OUTPUT FORMAT:

You must output a strictly structured JSON response containing:

- "executive_summary": A high-level, brutally honest evaluation of the idea.
- "validation_status": ["Green", "Amber", "Red"] with an explicit logical breakdown based on sub-agent data.
- "failure_case_study": Relevant historical examples if high risk is detected.
- "compiled_dashboard": The combined parsed data of all 4 sub-agents.
- "verified_resources": Array of all crawled valid resource URLs.

Internal Tooling & Execution Engine

- State Manager:** Loads and reconciles the full workspace state machine (pending_validation → running → compiled → delivered) from Supabase; guarantees the Orchestrator never synthesizes from a partially-written record.
- Prompt Builder:** Dynamically assembles the synthesis prompt by interpolating all four sub-agent JSON payloads plus the raw onboarding record into the base system prompt above, with token-budget truncation rules for oversized competitor lists.
- Failure Case-Study Vector Matcher:** A lightweight embedding index (pgvector on Supabase) of ~40-60 curated historical startup post-mortems, tagged by structural failure category (logistics-cost-collapse, overexpansion, hardware-margin-death, premature-scaling). The Orchestrator embeds the detected risk pattern and retrieves top-k matches above a similarity threshold before citing them.

Strict Output JSON Schema

```
{
  "executive_summary": "string",
  "validation_status": "Green | Amber | Red",
  "validation_reasoning": [
    { "signal": "string", "source_agent": "string", "weight": "high | medium | low" }
  ],
  "low_information_warning": {
    "triggered": "boolean",
    "affected_fields": ["string"],
    "confidence_penalty": "number"
  },
  "failure_case_study": [
    { "company": "string", "collapse_reason": "string", "matched_risk_pattern": "string", "similarity_score": "number" }
  ],
  "compiled_dashboard": {
    "market_research": "object",
    "product_strategy": "object",
    "pricing": "object",
    "marketing": "object"
  },
  "verified_resources": [
    { "url": "string", "source_agent": "string", "description": "string" }
  ]
}
```

3.2 Market Research Agent

- LIVE INTELLIGENCE
- TREND VALIDATION
- COMPETITOR SCRAPING

Functional Scope

Grounds the entire pipeline in observable reality. Establishes what competitors already exist, what users complain about, and how large the addressable market plausibly is — before any product or pricing decisions are made downstream.

Complete Input Matrix

Field	Wizard Step	Type
Core Idea	Step 1	Free text
Target Industry	Step 1	Enum / free text
Target Location	Step 1	Geo string
Target Audience Demographics	Step 2	Structured (age range, income band, segment)
Known Competitors	Step 2	Array of strings (optional)

System Prompt (Production)

You are an expert Market Research Analyst specializing in global tech industries, competitor intelligence, and market trend tracking. Your primary tool is SerpApi for searching real-time search engine results pages (SERPs).

INPUTS PROVIDED: Core Idea, Target Industry, Target Location, Target Audience Demographics, Known Competitors.

CRITICAL OPERATIONAL RULES:

- RAW SERP DATA EXTRACTION: Use SerpApi to scrape organic results, local listings, and global trends regarding the target niche.
- COMPETITOR ANALYTICS: Identify at least 3 direct and indirect competitors in the target location. Map out their features and user complaints (weaknesses) scraped from review platforms or public forums.
- SOURCE LINK RETENTION: You must capture and retain the exact source URLs for every competitor pricing page, news report, or industry benchmark you reference. Pass these URLs directly to the Orchestrator.
- MARKET SIZE ESTIMATION: Define the target market scope (TAM, SAM, SOM) conceptually using recent trend reports matching the user's demographic criteria.

EXPECTED OUTPUT FORMAT:

Output your results in a clear JSON matrix detailing:

- "market_trends": Current industry direction, tailwinds, and headwinds.
- "competitor_analysis": An array of objects containing competitor name, direct/indirect status, strengths, weaknesses, and source_url.
- "customer_pain_points": Aggregated user frustrations based on competitor analysis.
- "raw_sources": Array of all crawled valid resource URLs.

Internal Tooling & Execution Engine

- SerpApi Execution Layer:** Server-side calls to SerpApi's Organic Search, Google Trends, and Local Pack endpoints,

parameterized by Target Industry + Location. Results are deduplicated and ranked by domain authority heuristics before being passed into the LLM context.

- **Structured URL Retention Script:** A post-processing step that walks every LLM-cited claim and verifies the paired URL actually appeared in the raw SerpApi response set (prevents hallucinated citations from reaching the Orchestrator's "verified_resources").
- **Review/Forum Scraper (secondary):** Lightweight scraping utility targeting public review aggregators (e.g., G2, Trustpilot, Reddit threads surfaced via SerpApi) to extract verbatim complaint snippets for customer_pain_points.

Strict Output JSON Schema

```
{
  "market_trends": {
    "summary": "string",
    "tailwinds": ["string"],
    "headwinds": ["string"]
  },
  "market_size_estimate": {
    "tam": "string",
    "sam": "string",
    "som": "string",
    "methodology_note": "string"
  },
  "competitor_analysis": [
    {
      "name": "string",
      "type": "direct | indirect",
      "strengths": ["string"],
      "weaknesses": ["string"],
      "pricing_snapshot": "string",
      "source_url": "string"
    }
  ],
  "customer_pain_points": [
    { "pain_point": "string", "frequency_signal": "high | medium | low", "evidence_source_url": "string" }
  ],
  "data_confidence": "high | medium | low",
  "raw_sources": ["string"]
}
```

3.3 Product Strategist Agent

- UVP DEFINITION
- MVP SCOPING
- DESIGN TOKENS

Functional Scope

Converts raw market signal into a single, defensible Unique Value Proposition and a ruthlessly minimal MVP feature set — refusing feature bloat by forcing every proposed feature to trace back to the single highest-friction pain point identified in Market Research.

Complete Input Matrix

Field	Source	Type
Market Research Agent output (pain points, competitors)	Pipeline	JSON
Must-Have Feature	Wizard Step 3	Free text
Delivery Mechanism	Wizard Step 3	Enum (Web / Mobile / API / Hybrid)
Visual Theme Vibe — Style	Wizard Step 3	Enum
Visual Theme Vibe — Mode	Wizard Step 3	Enum (Light / Dark / Auto)
Visual Theme Vibe — Color Palette	Wizard Step 3	Structured color tokens

System Prompt (Production)

You are an elite Product Strategist and UX Architect. Your job is to translate complex market research data into a laser-focused product roadmap, defining the Unique Value Proposition (Value Prop) and the Minimum Viable Product (MVP).

INPUTS PROVIDED: Market Research Data (Pain points, Competitors), User Input (Core feature, Delivery mechanism, Visual Vibe [Style, Mode, Color Palette]).

CRITICAL OPERATIONAL RULES:
1. CORE PAIN POINT ANCHORING: Identify the single most painful friction point from the Market Research data. The entire

product must pivot around solving this one problem.

2. VALUE PROPOSITION GENERATION: Write a sharp, punchy Value Proposition that clearly tells the customer how this product fixes their life and why it is better than competitors.

3. MVP FEATURE FILTERING: Build a highly constrained feature matrix. You must divide ideas into:

- Must-Have (Core functionality required to fulfill the Value Prop. Keep this to an absolute minimum).
- Nice-to-Have (Fringe features, AI recommendations, rewards, etc., marked for post-launch).

4. BRAND VISUAL SYSTEM: Synthesize the user's aesthetic preferences (e.g., Minimalist Dark Mode with Finance Green) into a precise UI style guide summary for design execution.

EXPECTED OUTPUT FORMAT:

Output a structured JSON containing:

- "value_proposition": A clear statement explaining the unfair advantage.
- "mvp_features": {"must_have": [...], "nice_to_have": [...]}.
- "ui_vibe_specification": Visual theme parameters, styling constraints, and design tokens layout.

Internal Tooling & Execution Engine

- **Feature Matrix Categorization Engine:** A constrained-generation step (structured output via function-calling) that forces the LLM to bucket every proposed capability into exactly one of must_have / nice_to_have, with a hard cap (default: 5) on must_have items to enforce MVP discipline.
- **Visual Styling Design Tokens Mapper:** Translates free-text aesthetic descriptions into a normalized design-token object (color hex values, font-pairing suggestion, spacing scale) directly consumable by the Next.js Tailwind theme layer for downstream prototype generation.

Strict Output JSON Schema

```
{
  "core_pain_point_anchor": "string",
  "value_proposition": "string",
  "mvp_features": {
    "must_have": [ { "feature": "string", "justification": "string" } ],
    "nice_to_have": [ { "feature": "string", "rationale": "string" } ]
  },
  "ui_vibe_specification": {
    "style": "string",
    "mode": "light | dark | auto",
    "color_palette": { "primary": "#hex", "secondary": "#hex", "accent": "#hex", "background": "#hex" },
    "font_pairing_suggestion": "string",
    "design_tokens_note": "string"
  }
}
```

3.4 Pricing Agent

UNIT ECONOMICS LTV:CAC TIERED SAAS PRICING

Functional Scope

Removes guesswork from pricing by running actual quantitative unit-economics formulas against the founder's declared cost inputs and acquisition assumptions, then generating a defensible three-tier SaaS pricing ladder.

Complete Input Matrix

Field	Source	Type
MVP Features List	Product Strategist output	JSON array
Expected Initial Tech Stack	Wizard Step 4	Free text / structured
Target Customer Segment	Wizard Step 4	Enum (B2C / SMB / Mid-Market / Enterprise)
Customer Acquisition Plan (CAC Input)	Wizard Step 4	Numeric estimate
Minimum Profit Expectation	Wizard Step 4	Numeric / percentage

System Prompt (Production)

You are a Financial Engineer and B2B/B2C SaaS Pricing Specialist. Your toolset includes a Python calculation engine to execute precise quantitative formulas. You do not generate arbitrary numbers; you compute pricing models based on operational reality.

INPUTS PROVIDED: MVP Features List, Tech Stack Expectations, Target Customer Segment, Customer Acquisition Plan (CAC), Minimum Profit Expectation.

CRITICAL OPERATIONAL RULES:

1. BASE COGS CALCULATIONS: Calculate the estimated Per-User Cost of Goods Sold (COGS) based on infrastructure components (e.g., API token consumption, database storage, hosting overhead).

2. LTV/CAC RATIO ENFORCEMENT: Factor in the target customer segment to run the standard SaaS unit economics calculation. Ensure the estimated Lifetime Value (LTV) maps to a minimum of 3x the projected Customer Acquisition Cost (CAC).

3. THREE-TIER STRUCTURE GENERATION: Compute a three-tier pricing structure (Starter, Pro, Enterprise) optimized for high conversion and profit sustainability:

- Starter Tier: Lower computing cost features only, meant for user acquisition hook.

- Pro Tier (Sweet Spot): Standard tier capturing the majority of revenue with a target gross margin of 60% - 70%.

- Enterprise Tier: Flat rate or custom quoting for high-compute or advanced features.

EXPECTED OUTPUT FORMAT:

Return a JSON model payload with:

- "estimated_unit_economics": {"per_user_cogs": X, "target_ltv_cac_ratio": Y}.

- "pricing_tiers": {"starter": {...}, "pro": {...}, "enterprise": {...}}.

- "roi_projection": Percentage yield and estimated timeline to hit the minimum profit expectation.

Internal Tooling & Execution Engine

- **Python Calculation Engine:** A sandboxed function-calling tool (invoked by the LLM via tool-use, not free-form arithmetic) that executes deterministic COGS, LTV, CAC-payback, and gross-margin formulas server-side, returning numeric results the LLM must incorporate verbatim rather than re-deriving.
- **Unit Economics Simulator:** Runs a small Monte-Carlo-style sensitivity sweep (±20% on CAC and churn assumptions) to attach a confidence range to the ROI projection rather than a single point estimate.
- **SaaS 3:1 LTV/CAC Rule Checking Engine:** A hard-coded business-rule validator that flags `ltv_cac_ratio < 3.0` as a structural red flag surfaced directly to the Orchestrator's `validation_status` computation.

Strict Output JSON Schema

```
{
  "estimated_unit_economics": {
    "per_user_cogs": "number",
    "estimated_ltv": "number",
    "estimated_cac": "number",
    "target_ltv_cac_ratio": "number",
    "ltv_cac_rule_passed": "boolean"
  },
  "pricing_tiers": {
    "starter": { "price": "number", "billing_cycle": "string", "included_features": ["string"] },
    "pro": { "price": "number", "billing_cycle": "string", "included_features": ["string"], "target_gross_margin_pct": "number" },
    "enterprise": { "pricing_model": "flat_rate | custom_quote", "included_features": ["string"] }
  },
  "roi_projection": {
    "projected_yield_pct": "number",
    "months_to_min_profit_target": "number",
    "confidence_range": { "low": "number", "high": "number" }
  }
}
```

3.5 Marketing Agent

GTM CHANNELS

COPYWRITING

AUTOMATION PAYLOAD

Functional Scope

Converts the validated product and pricing strategy into an executable go-to-market plan: which channels are worth the founder's limited budget, what the brand says, and ready-to-post copy formatted per-platform.

Complete Input Matrix

Field	Source	Type
MVP Strategy / Value Prop	Product Strategist output	JSON
Pricing Tiers	Pricing Agent output	JSON
Tone of Voice	Wizard Step 5	Enum (Professional / Playful / Bold / Minimal)
Marketing Budget Size	Wizard Step 5	Numeric range
Launch Timeline	Wizard Step 5	Date / duration

System Prompt (Production)

You are a Growth Marketing Director and Copywriting specialist. Your job is to build Go-To-Market (GTM) distribution channels, write high-converting taglines, and write channel-specific promotional content.

INPUTS PROVIDED: MVP Strategy, Core Value Prop, Target Demographics, Tone of Voice, Budget Size, Launch Timeline.

CRITICAL OPERATIONAL RULES:

1. DISTRIBUTION CHANNEL MAPPING: Filter and prioritize the top 3 high-leverage marketing platforms based on budget and target demographics (e.g., Prioritize LinkedIn/Twitter/Product Hunt for B2B SaaS; Prioritize Instagram/TikTok for B2C E-commerce).
2. TAGLINE CREATION: Generate 3 variations of high-converting, punchy taglines using the specified brand Tone of Voice.
3. CHANNEL-SPECIFIC COPYWRITING: Write ready-to-use sample posts optimized for the distinct algorithm rules of each chosen channel (e.g., 280-character limit threads for X, bulleted professional formatting for LinkedIn, community-first informal posts for Reddit).
4. OUTBOUND/AUTOMATION TRIGGER: Format the data payload into a clean, standardized JSON format so that it can be seamlessly forwarded to an external orchestration or automation platform (like an n8n webhook connection for distribution).

EXPECTED OUTPUT FORMAT:

Output a structured JSON outlining:

- "recommended_channels": Ordered array of the top 3 platforms with explicit strategic reasoning.
- "brand_taglines": Array of text variations.
- "sample_campaign_posts": Array of objects containing {"channel_name": X, "post_content": Y, "visual_asset_prompt": Z}.
- "automation_payload": Clean stringified sub-object ready for external API shipping.

Internal Tooling & Execution Engine

- **Channel Priority Mapping Arrays:** A lookup matrix cross-referencing Target Customer Segment × Budget Band × Demographics against a maintained ranked-channel table (updated quarterly), which seeds the LLM's channel selection rather than letting it invent priorities unconstrained.
- **Formatting Wrappers:** Per-channel post-processing enforcing platform constraints — X/Twitter 280-character hard truncation with thread-splitting, LinkedIn Markdown-to-rich-text bullet conversion, Reddit informal-tone linter.
- **Outbound n8n HTTP Webhook Integration Slot:** The `automation_payload` field is shipped, on user opt-in, to a configurable n8n webhook URL stored in workspace settings, enabling one-click forwarding into the founder's own external automation/distribution stack.

Strict Output JSON Schema

```
{
  "recommended_channels": [
    { "channel": "string", "priority_rank": "number", "strategic_reasoning": "string", "estimated_monthly_cost": "number" }
  ],
  "brand_taglines": ["string", "string", "string"],
  "sample_campaign_posts": [
    { "channel_name": "string", "post_content": "string", "visual_asset_prompt": "string" }
  ],
  "automation_payload": {
    "format_version": "string",
    "payload_json_stringified": "string",
    "webhook_ready": "boolean"
  }
}
```

4. Technical Architecture, Schema & Folder Tree

4.1 Supabase PostgreSQL Schema (DDL)

```
-- =====
-- USERS (managed by Supabase Auth; extended profile table)
-- =====
create table public.profiles (
  id          uuid primary key references auth.users(id) on delete cascade,
  full_name   text,
  avatar_url  text,
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);

-- =====
-- WORKSPACES (one per startup-idea validation run)
-- =====
create table public.workspaces (
  id          uuid primary key default gen_random_uuid(),
  owner_id    uuid not null references auth.users(id) on delete cascade,
  title       text not null default 'Untitled Idea',
  status      text not null default 'draft'
              check (status in ('draft','pending_validation','running',
                                'compiled','delivered','failed')),
  is_unlocked boolean not null default false,
  stripe_payment_intent_id text,
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);

-- =====
-- ONBOARDING_INPUTS (wizard state, one row per workspace)
-- =====
create table public.onboarding_inputs (
  id          uuid primary key default gen_random_uuid(),
  workspace_id uuid not null references public.workspaces(id) on delete cascade,
  core_idea   text,
  target_industry text,
  target_location text,
  audience_demographics jsonb,
  known_competitors jsonb,
  must_have_feature text,
  delivery_mechanism text
              check (delivery_mechanism in ('web','mobile','api','hybrid')),
  visual_vibe  jsonb,
  tech_stack   jsonb,
  customer_segment text
              check (customer_segment in ('b2c','smb','mid_market','enterprise')),
  cac_input    numeric,
  min_profit_expectation numeric,
  tone_of_voice text,
  marketing_budget numeric,
  launch_timeline date,
  updated_at   timestamptz not null default now(),
  unique(workspace_id)
);

-- =====
-- AGENT_RUNS (one row per sub-agent execution, append-only log)
-- =====
create table public.agent_runs (
  id          uuid primary key default gen_random_uuid(),
  workspace_id uuid not null references public.workspaces(id) on delete cascade,
  agent_name  text not null
              check (agent_name in ('orchestrator','market_research',
                                    'product_strategist','pricing','marketing')),
  status      text not null default 'queued'
              check (status in ('queued','running','succeeded','failed','degraded')),
  input_payload jsonb,
  output_payload jsonb,
  error_message text,
  retry_count  integer not null default 0,
  started_at   timestamptz,
  completed_at timestamptz,
  created_at   timestamptz not null default now()
);

-- =====
-- RESOURCE_LINKS (verified URLs surfaced by Market Research / Orchestrator)
-- =====
create table public.resource_links (
  id          uuid primary key default gen_random_uuid(),
  workspace_id uuid not null references public.workspaces(id) on delete cascade,
  url         text not null,
  source_agent text not null,
```

```

description      text,
created_at       timestampz not null default now()
);

-- =====
-- INDEXES
-- =====
create index idx_workspaces_owner      on public.workspaces(owner_id);
create index idx_agent_runs_workspace on public.agent_runs(workspace_id);
create index idx_resource_links_workspace on public.resource_links(workspace_id);

-- =====
-- ROW LEVEL SECURITY
-- =====
alter table public.workspaces      enable row level security;
alter table public.onboarding_inputs enable row level security;
alter table public.agent_runs      enable row level security;
alter table public.resource_links  enable row level security;

create policy "Owners can manage their workspaces"
  on public.workspaces for all
  using (auth.uid() = owner_id);

create policy "Owners can access their onboarding inputs"
  on public.onboarding_inputs for all
  using (
    workspace_id in (select id from public.workspaces where owner_id = auth.uid())
  );

create policy "Owners can read their agent runs"
  on public.agent_runs for select
  using (
    workspace_id in (select id from public.workspaces where owner_id = auth.uid())
  );

create policy "Owners can read their resource links"
  on public.resource_links for select
  using (
    workspace_id in (select id from public.workspaces where owner_id = auth.uid())
  );

```

4.2 Backend Folder Structure (FastAPI + LangChain)

```

backend/
├── app/
│   ├── main.py           # FastAPI app entrypoint, CORS, router mounting
│   ├── config.py         # Pydantic Settings (env vars, secrets)
│   ├── dependencies.py   # Supabase client, auth guard, rate limiter DI
│   └── api/
│       ├── v1/
│       │   ├── router.py # Aggregates all v1 routers
│       │   ├── workspaces.py # CRUD: create/read workspace
│       │   ├── onboarding.py # Wizard step PATCH endpoints
│       │   ├── pipeline.py  # POST /pipeline/run, GET /pipeline/status
│       │   ├── payments.py  # Stripe webhook + payment-intent creation
│       │   └── resources.py # GET verified resource links per workspace
│       └── agents/
│           ├── base_agent.py # Shared LLM client wrapper, retry/backoff logic
│           ├── orchestrator_agent.py # Chief Orchestrator implementation
│           ├── market_research_agent.py
│           ├── product_strategist_agent.py
│           ├── pricing_agent.py
│           ├── marketing_agent.py
│           └── prompts/
│               ├── orchestrator_prompt.py
│               ├── market_research_prompt.py
│               ├── product_strategist_prompt.py
│               ├── pricing_prompt.py
│               └── marketing_prompt.py
│       └── tools/
│           ├── serpapi_client.py # SerpApi wrapper (organic, trends, local)
│           ├── url_retention.py  # Citation verification against raw SERP set
│           ├── pricing_calculator.py # Deterministic COGS/LTV/CAC formulas
│           ├── unit_economics_sim.py # Sensitivity sweep simulator
│           ├── failure_case_vector_store.py # pgvector similarity search
│           ├── channel_priority_matrix.py
│           ├── formatting_wrappers.py # Per-channel character-limit/format rules
│           └── n8n_webhook_client.py
│       └── models/
│           └── schemas.py # Pydantic request/response models

```

```

├── agent_output_schemas.py # Strict output schemas per agent (§3)
├── db_models.py             # ORM/typed row models mirroring $4.1 DDL
├── orchestration/
│   ├── pipeline_runner.py   # Async task graph: sequential + parallel stages
│   ├── state_manager.py    # Workspace status transitions
│   └── error_boundary.py    # Retry, degraded-mode fallback logic (§2.3)
├── db/
│   ├── supabase_client.py
│   └── migrations/         # SQL migration files (see $4.1)
├── workers/
│   └── pipeline_worker.py   # Render background worker entrypoint
├── tests/
│   ├── test_agents/
│   ├── test_pipeline/
│   └── test_api/
├── requirements.txt
├── Dockerfile
└── render.yaml             # Render service + worker definitions

```

4.3 Frontend Folder Structure (Next.js 14 App Router)

```

frontend/
├── app/
│   ├── layout.tsx
│   ├── page.tsx             # Marketing landing page
│   ├── globals.css
│   ├── (auth)/
│   │   ├── login/page.tsx
│   │   └── signup/page.tsx
│   ├── onboarding/
│   │   ├── layout.tsx       # Wizard shell + progress indicator
│   │   ├── step-1-idea/page.tsx
│   │   ├── step-2-audience/page.tsx
│   │   ├── step-3-product/page.tsx
│   │   ├── step-4-pricing/page.tsx
│   │   └── step-5-marketing/page.tsx
│   ├── dashboard/
│   │   ├── page.tsx         # Workspace list
│   │   └── [workspaceId]/
│   │       ├── page.tsx     # Compiled dashboard view
│   │       ├── loading.tsx  # Skeleton while pipeline runs
│   │       ├── reality-check.tsx # Orchestrator banner + failure case study
│   │       ├── market-panel.tsx
│   │       ├── product-panel.tsx
│   │       ├── pricing-panel.tsx
│   │       └── marketing-panel.tsx
│   ├── checkout/
│   │   └── [workspaceId]/page.tsx # Stripe checkout for $20 unlock
│   └── api/
│       └── webhooks/stripe/route.ts # Next.js edge route → forwards to FastAPI
├── components/
│   ├── ui/                  # shadcn/ui primitives
│   ├── wizard/
│   │   ├── WizardStepper.tsx
│   │   └── fields/          # Reusable form field components
│   ├── dashboard/
│   │   ├── ValidationStatusBadge.tsx
│   │   ├── ResourceLinkList.tsx
│   │   └── AgentPayloadCard.tsx
│   └── shared/
│       ├── Navbar.tsx
│       └── Footer.tsx
├── lib/
│   ├── supabase/
│   │   ├── client.ts
│   │   └── server.ts
│   ├── api.ts              # Typed fetch wrappers to FastAPI backend
│   ├── zod-schemas.ts     # Wizard + agent-payload validation schemas
│   └── realtime.ts         # Supabase Realtime subscription hooks

```

```
| hooks/
|   | useWorkspace.ts
|   | usePipelineStatus.ts
|   | useOnboardingForm.ts
|
| types/
|   | agent-outputs.d.ts          # Mirrors $3 JSON schemas
|
| public/
| tailwind.config.ts
| next.config.mjs
| package.json
| vercel.json
```

5. Non-Functional Requirements & Security

5.1 Performance

- Full pipeline (5 agent stages) target completion: < 90 seconds p50, < 150 seconds p95, executed asynchronously via Render worker so the client never holds an open HTTP connection.
- Dashboard should reflect stage-by-stage progress via Supabase Realtime, not a single blocking spinner.

5.2 Security & Compliance

- All sub-agent LLM calls run server-side only; no LLM API keys are ever exposed to the Next.js client bundle.
- Supabase RLS enforced on every table containing user-owned data (§4.1); service-role key used only inside the FastAPI backend, never in frontend code.
- Stripe webhook signature verification required before flipping `is_unlocked`.
- PII in `onboarding_inputs` (demographics) treated as sensitive; encrypted at rest via Supabase's default column encryption where applicable, and excluded from any analytics export.

5.3 Observability

- Every `agent_runs` row captures input/output payloads for replay/debugging and post-hoc prompt evaluation.
- Structured logging (JSON logs) from FastAPI shipped to Render's log drains; critical pipeline failures alert via a configured webhook (Slack/email).

Open Risk: SerpApi and LLM provider rate limits are external dependencies outside the platform's control; the degraded-mode fallback in §2.3 is a mitigation, not an elimination of this risk, and should be monitored post-launch.

6. Appendix — Glossary & Assumptions

Term	Definition
TAM / SAM / SOM	Total Addressable Market / Serviceable Addressable Market / Serviceable Obtainable Market — standard nested market-sizing framework.
LTV / CAC	Customer Lifetime Value / Customer Acquisition Cost — the ratio the Pricing Agent enforces at a minimum of 3:1 per standard SaaS unit-economics practice.
COGS	Cost of Goods Sold — here, the per-user marginal infrastructure cost (LLM tokens, hosting, storage) of serving the product.
RLS	Row-Level Security — Postgres/Supabase mechanism restricting row visibility to the authenticated owner.
GTM	Go-To-Market — the channel and messaging plan for launch, owned by the Marketing Agent.

Key Assumptions

- MVP monetization is a single \$20 transaction per validation run, not a subscription; subscription tiers are explicitly out of scope for v1 and noted here as a candidate v2 expansion.
- SerpApi is the sole live-data provider for Market Research at launch; additional data providers (e.g., Similarweb, Crunchbase) are a future integration surface, not required for MVP.
- The historical failure-case vector index is a curated, manually maintained dataset at launch — not a live-scraped corpus — to guarantee citation accuracy.